

A3 – PLAN DE TESTS UNITAIRES

POC Signature Editor —Stage NeoLedge

Table des matières

1. Objectif du plan de tests	1
2. Contexte.....	1
3. Démarche de préparation des jeux de tests	1
3.1 Inventaire des fonctions et de leurs contrats	2
3.2 Catégorisation systématique des cas.....	2
3.3 Cas spécifiques à la restauration JSON	2
3.4 Organisation des suites.....	2
4. Plan de tests détaillé.....	2
4.1 Suite 1 — Fonctions de conversion et de validation.....	2
4.2 Suite 2 — Construction du payload d'export (buildExportPayload)	4
4.3 Suite 3 — Parsing et restauration JSON (parseSignatureJson)	4
4.4 Suite 4 — Persistance des signatures (localStorage)	5
5. Jeux de données de test reproductibles	5
6. Résultats d'exécution.....	6

1. Objectif du plan de tests

Ce document formalise le plan de tests unitaires établi pour les fonctions utilitaires du POC Signature Editor : conversion et validation de types, construction et restauration du payload d'export JSON, et persistance des signatures sauvegardées. Il couvre pour chaque fonction les cas nominaux, les cas limites et les cas d'erreur, présente des jeux de données reproductibles, et documente les résultats réels d'exécution obtenus avec le framework Vitest.

2. Contexte

Le POC Signature Editor repose sur un ensemble de fonctions utilitaires pures qui traitent les données de signature : conversion de types, validation des polices et alignements, construction du format d'export JSON et persistance des signatures sauvegardées en localStorage. Ces fonctions constituent le cœur de la logique métier de l'éditeur — une erreur silencieuse sur l'une d'elles peut produire un export corrompu ou un comportement inattendu sans message d'erreur visible pour l'utilisateur.

La préparation des jeux de tests a donc consisté à modéliser exhaustivement les comportements attendus et non attendus de chaque fonction, avant même d'écrire les assertions, afin d'identifier les chemins d'exécution qui pourraient provoquer un dysfonctionnement.

3. Démarche de préparation des jeux de tests

3.1 Inventaire des fonctions et de leurs contrats

La première étape a été d'identifier, pour chaque fonction utilitaire, son contrat exact : quelles entrées sont valides, lesquelles sont tolérées avec une valeur par défaut, lesquelles doivent lever une erreur explicite. Huit unités ont été couvertes par le plan de tests : `asString`, `asNumber`, `asBoolean`, `toFontChoice` et `isAlignment` (fonctions de normalisation), `buildExportPayload` et `parseSignatureJson` (construction et restauration du format d'échange JSON), et la couche de persistance `localStorage` (`signature_saved_cards`, `signature_next_id`).

3.2 Catégorisation systématique des cas

Pour chaque fonction, trois catégories de cas ont été préparées :

Cas nominaux : entrées valides et représentatives du fonctionnement normal — par exemple `asNumber(14, 12)` doit retourner 14, ou `toFontChoice('Dancing Script')` doit retourner 'Dancing Script'.

Cas limites : valeurs aux frontières du domaine d'entrée — chaîne vide, zéro, valeur décimale, tableau JSON vide {}, absence de champs optionnels.

Cas d'erreur / entrées inattendues : `null`, `undefined`, un nombre à la place d'une chaîne, un JSON malformé ou un type racine inattendu (tableau, chaîne, nombre). L'objectif est de vérifier que le logiciel ne produit jamais de comportement silencieusement incorrect face à une entrée hors contrat.

3.3 Cas spécifiques à la restauration JSON

La fonction `parseSignatureJson` a nécessité une attention particulière car elle constitue le point d'entrée de tout fichier externe réimporté par l'utilisateur : JSON malformé, racine non-objet (tableau, chaîne, nombre, `null` littéral), champs inconnus à ignorer sans lever d'erreur, et compatibilité ascendante entre le schéma v1 (sans texte complémentaire) et v2 (avec `topContent`/`bottomContent`). Chaque cas vérifie que la fonction retourne un état propre et prévisible plutôt que de planter silencieusement.

3.4 Organisation des suites

Les 64 cas ont été organisés en 4 suites correspondant aux 11 groupes de tests (`describe`) du fichier `signature.test.ts`, chacune couvrant un périmètre fonctionnel précis. Cette organisation permet à tout développeur rejoignant le projet de comprendre immédiatement quelle suite couvre quel périmètre, et d'ajouter de nouveaux cas sans risque de chevauchement.

4. Plan de tests détaillé

4.1 Suite 1 — Fonctions de conversion et de validation

`asString(value, defaultValue)`

Cas	Entrée / Scénario	Résultat attendu
Nominal	<code>asString('Arial', 'défaut')</code>	'Arial'
Nominal (limite)	<code>asString('', 'défaut')</code>	" (chaîne vide conservée)
Erreur	<code>asString(null, 'défaut')</code>	'défaut'
Erreur	<code>asString(undefined, 'défaut')</code>	'défaut'
Erreur	<code>asString(42, 'défaut')</code>	'défaut'
Erreur	<code>asString({}, 'défaut')</code>	'défaut'

`asNumber(value, defaultValue)`

Cas	Entrée / Scénario	Résultat attendu
Nominal	asNumber(14, 12)	14
Nominal	asNumber(1.5, 1)	1.5
Limite	asNumber(0, 12)	0
Limite	asNumber(Infinity, 12)	12
Erreur	asNumber(NaN, 12)	12
Erreur	asNumber('abc', 12)	12
Erreur	asNumber(null, 12)	12 (attendu) — voir §6, anomalie détectée
Erreur	asNumber(undefined, 12)	12

asBoolean(value, defaultValue)

Cas	Entrée / Scénario	Résultat attendu
Nominal	asBoolean(true, false)	true
Nominal	asBoolean(false, true)	false
Erreur	asBoolean('true', false)	false (le défaut ; seul le type boolean est accepté)
Erreur	asBoolean(1, false)	false
Erreur	asBoolean(null, true)	true

toFontChoice(value)

Cas	Entrée / Scénario	Résultat attendu
Nominal	toFontChoice('Arial')	'Arial'
Nominal	toFontChoice('Dancing Script')	'Dancing Script'
Nominal	toFontChoice('Pacifico')	'Pacifico'
Nominal	toFontChoice('Times New Roman')	'Times New Roman'
Nominal	toFontChoice('Courier New')	'Courier New'
Erreur	toFontChoice('Comic Sans')	'Arial' (police inconnue → repli)
Erreur	toFontChoice(null)	'Arial'
Limite	toFontChoice('')	'Arial'

isAlignment(value)

Cas	Entrée / Scénario	Résultat attendu
Nominal	isAlignment('left')	'left'
Nominal	isAlignment('center')	'center'
Nominal	isAlignment('right')	'right'
Erreur	isAlignment('justify')	'left' (valeur hors domaine → repli)
Erreur	isAlignment(null)	'left'
Erreur	isAlignment(undefined)	'left'

4.2 Suite 2 — Construction du payload d'export (buildExportPayload)

Version v1 — sans texte complémentaire

Cas	Entrée / Scénario	Résultat attendu
Nominal	topContent et bottomContent absents	payload.version === 'v1'
Limite	topContent et bottomContent vides ('')	payload.version === 'v1'
Nominal	name, initials, fontFamily, fontSize, fontColor, showBorder renseignés	tous les champs restitués à l'identique
Nominal	génération du payload	exportedAt au format ISO 8601
Limite	structure v1	topContent / bottomContent absents de l'objet (undefined)

Version v2 — avec texte complémentaire

Cas	Entrée / Scénario	Résultat attendu
Nominal	topContent renseigné	payload.version === 'v2'
Limite	seul bottomContent renseigné (topContent omis)	payload.version === 'v2'
Nominal	topContent='Chef de projet', bottomContent='NeoLedge'	champs restitués à l'identique
Nominal	topAlignment='center', bottomAlignment='right'	alignements restitués à l'identique
Limite	topAlignment / bottomAlignment absents	'left' appliqué par défaut aux deux zones

4.3 Suite 3 — Parsing et restauration JSON (parseSignatureJson)

Cas nominaux

Cas	Entrée / Scénario	Résultat attendu
Nominal	JSON v1 valide complet	name, fontFamily et version restitués
Nominal	JSON v2 valide avec texte complémentaire	topContent, bottomContent, topAlignment restitués
Nominal	round-trip export → JSON.stringify → parse	objet réimporté identique à l'original

Valeurs par défaut (cas limites)

Cas	Entrée / Scénario	Résultat attendu
Limite	objet vide '{}'	name/initials='', fontFamily='Arial', fontSize=14, fontColor='#000000', showBorder=false
Limite	fontFamily: 'Comic Sans MS'	corrigé en 'Arial'
Limite	version v2, topAlignment: 'justify'	corrigé en 'left'
Limite	fontSize: 'grande' (non numérique)	corrigé en 14
Limite	showBorder: 'oui' (non booléen)	corrigé en false

Cas d'erreur

Cas	Entrée / Scénario	Résultat attendu
Erreur	JSON malformé '{invalid json}'	lève une erreur "JSON invalide"
Erreur	racine = tableau '[1, 2, 3]'	lève une erreur
Erreur	racine = null littéral 'null'	lève une erreur
Erreur	racine = chaîne '"texte simple"'	lève une erreur
Erreur	racine = nombre '42'	lève une erreur
Erreur	chaîne vide ''	lève une erreur
Limite	champs inconnus (champsInconnu, autreChamp)	ignorés sans erreur

4.4 Suite 4 — Persistance des signatures (localStorage)

Cas	Entrée / Scénario	Résultat attendu
Nominal	sauvegarde d'un tableau de cartes dans signature_saved_cards	lecture ultérieure retourne le même tableau (1 élément, name conservé)
Limite	clé signature_saved_cards absente	lecture retourne un tableau vide []
Nominal	incrémementation de signature_next_id	la valeur stockée passe de 1 à 2
Nominal	suppression d'une carte par id (3 cartes → 2)	la carte ciblée n'est plus présente, les 2 autres restent
Nominal	remplacement d'une carte existante (id=1)	le nom de la carte est mis à jour, l'id est conservé
Limite	10 cartes stockées en parallèle	les 10 éléments sont conservés et restituables

5. Jeux de données de test reproductibles

Pour garantir la reproductibilité des tests (aucune valeur aléatoire, aucune dépendance à l'horloge système hors du champ exportedAt), les jeux de données suivants sont utilisés comme fixtures fixes et réutilisables

Fixture v1 — signature simple

Cas	Entrée / Scénario	Résultat attendu
Donnée	name: 'Julien Gournay', initials: 'JG', fontFamily: 'Arial', fontSize: 14, fontColor: '#1F3864', showBorder: false	Utilisée pour valider la construction et le parsing du format v1

Fixture v2 — signature avec texte complémentaire

Cas	Entrée / Scénario	Résultat attendu
Donnée	name: 'Julien Gournay', initials: 'JG', fontFamily: 'Arial', fontSize: 14, fontColor: '#000000', showBorder: true, topContent: 'Chef de projet', bottomContent: 'NeoLedge', topAlignment: 'center', bottomAlignment: 'right'	Utilisée pour valider la construction et le parsing du format v2

Fixture — persistance localStorage

Cas	Entrée / Scénario	Résultat attendu
Donnée	[{ id: 1, name: 'Carte 1' }, { id: 2, name: 'Carte 2' }, { id: 3, name: 'Carte 3' }]	Utilisée pour valider la suppression et le remplacement ciblés par id

Chaque suite localStorage réinitialise l'état via `beforeEach(() => localStorage.clear())`, garantissant l'isolation et la reproductibilité d'un lancement à l'autre, indépendamment de l'ordre d'exécution des tests.

6. Résultats d'exécution

Les 64 tests ont été exécutés avec Vitest 1.6 (environnement jsdom, nécessaire à la disponibilité de l'API localStorage). Résultat : 63 tests passants et 1 test en échec, en 11 ms d'exécution pure.

Anomalie détectée — `asNumber(null, 12)` : le test attend un repli sur la valeur par défaut (12) lorsque la valeur fournie est null, conformément au contrat documenté. Or l'implémentation utilise `Number(value)` puis `isFinite(n)` : `Number(null)` valant 0 (un nombre fini), la fonction retourne 0 au lieu du défaut. Ce cas limite, non couvert par l'implémentation initiale, illustre concrètement l'utilité de la préparation exhaustive des jeux de tests décrite en section 3 : il a été identifié avant toute mise en production grâce au test dédié, et non découvert a posteriori par un utilisateur.

Correctif proposé : ajouter une vérification explicite du type (`value === null || value === undefined`) avant la conversion, plutôt que de s'appuyer sur la coercition implicite de `Number()`.